Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas

Bank usac

PONDERACIÓN: 21 pts

Tiempo estimado: 50 hrs

Índice

Contenido

1. MARCO FORMATIVO 3
1.1. Valor 3
1.2. Competencia(s) 3
1.3. Habilidad(es) blandas a formar 3
2. Resultado del Aprendizaje 3
2.1. Objetivo SMART 3
3. Resumen Ejecutivo 4
4. Enunciado del Proyecto 4
4.1 Descripción del problema o necesidad a resolver 4
Extensión Funcional del Sistema 4
4.2 Alcance del proyecto 8
4.3 Recursos y herramientas a utilizar 8
4.4 Entregables 9
5. Material de apoyo 9
6. Metodología 10
7. Cronograma 11
8. Rúbrica de Calificación 12
8.1 Requisitos para optar a la calificación 12
8.2 Detalle de la Calificación 13
8.3 Comentarios Generales 18

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en el proyecto?
Responsabilidad compartida (Ownership)	Cada integrante es responsable del ciclo completo de su microservicio: desarrollo, despliegue, automatización (CI/CD) e infraestructura (Terraform), alineado a prácticas DevOps y SRE.

1.2. Competencia(s)

Diseñar, automatizar y desplegar un sistema distribuido en la nube utilizando prácticas de **DevOps, CI/CD e Infraestructura como Código (IaC)**, garantizando confiabilidad, escalabilidad y mantenibilidad.

1.3. Habilidad(es) blandas a formar

- Trabajo colaborativo en entornos distribuidos
- Toma de decisiones técnicas bajo restricciones reales
- Pensamiento crítico sobre automatización
- Responsabilidad sobre el ciclo de vida completo del software

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	**Medible (¿Cuánto?)**	**Alcanzable (¿Cómo?)**	**Realista (¿Para qué?)**	**A Tiempo (¿Cuándo?)**
Extender sistema distribuido con CI/CD y cloud	Pipeline funcional + despliegue en nube	Uso de Kubernetes + Terraform	Simulación de entorno productivo real	4 semanas

3. Resumen Ejecutivo

En esta fase se evoluciona el sistema bancario desarrollado previamente, incorporando prácticas de automatización, despliegue continuo e infraestructura como código.

El sistema será desplegado en la nube utilizando Kubernetes, con pipelines CI/CD funcionales y bases de datos provisionadas en máquinas virtuales mediante Terraform, manteniendo los principios de arquitectura asíncrona y desacoplada.

4. Enunciado del Proyecto

4.1 Descripción del problema o necesidad a resolver

El desarrollo de software moderno requiere no solo la implementación de funcionalidades, sino también la automatización de su integración, entrega y despliegue.
Las organizaciones necesitan sistemas capaces de:

- Integrarse continuamente
- Desplegarse automáticamente
- Escalar dinámicamente
- Mantener infraestructura reproducible

Extensión Funcional del Sistema

1. Customer Service
Nueva funcionalidad: Gestión de estado del cliente (KYC)
El sistema deberá permitir manejar el estado de validación de un cliente (Know Your Customer).
Capacidades

- Registrar estado del cliente:
  - PENDING
  - VERIFIED
  - REJECTED
- Actualizar estado del cliente
- Validar que solo clientes verificados puedan realizar transacciones

2. Account Service
Nueva funcionalidad: Tipos de cuenta y reglas de negocio
El sistema deberá soportar diferentes tipos de cuenta con comportamientos distintos.
Capacidades

- Crear cuentas:
  - Ahorro
  - Corriente
- Definir reglas:
  - Límite de saldo mínimo
  - Comisión por transacción (opcional)

Transaction Service
Nueva funcionalidad: Historial y trazabilidad de transacciones
El sistema deberá mantener un historial completo de transacciones.
Capacidades

- Consultar historial por cuenta
- Filtrar por:
  - Fecha
  - Estado
- Registrar estados intermedios:
  - PENDING
  - APPROVED
  - FAILED

Payment Service
Nueva funcionalidad: Simulación de pagos externos
El sistema deberá simular integración con sistemas externos de pago.
Capacidades

- Procesar pagos externos
- Simular respuestas:
  - Éxito
  - Fallo
  - Timeout

Notification & Audit Service
Nueva funcionalidad: Notificaciones inteligentes y auditoría extendida
El sistema deberá clasificar y registrar eventos con mayor detalle.
Capacidades

- Clasificar eventos:
  - INFO
  - WARNING
  - ERROR
- Generar notificaciones según tipo
- Almacenar historial de eventos

Funcionalidad transversal obligatoria
Gestión avanzada de eventos
Todos los servicios deben implementar:
1. Idempotencia
- Evitar duplicación de efectos ante eventos repetidos

2. CorrelationId
- Trazabilidad completa de cada flujo

3. Manejo de estados
- Cada operación debe tener estados definidos

4. Manejo de errores
- Fallos deben generar eventos claros

Integración con Saga
Las nuevas funcionalidades deben integrarse en el flujo de transferencia:
- Validación KYC antes de transferir
- Validación de tipo de cuenta
- Registro en historial
- Simulación de fallos externos
- Notificación final

Flujo de Integración y Despliegue Continuo (CI/CD)
El proceso inicia con la creación de una nueva funcionalidad en una rama de tipo:
- feature/<nombre-funcionalidad>

El estudiante deberá:
- Implementar los cambios necesarios en el microservicio correspondiente
- Realizar commits frecuentes
- Publicar los cambios mediante push al repositorio remoto

Ejecución del pipeline de Integración Continua (CI):
Al realizar un push sobre una rama feature, se ejecutará automáticamente el pipeline de CI, el cual deberá incluir:
Fases obligatorias
- **Build:** Compilación del proyecto
- **Test:** Ejecución de pruebas automatizadas
- **Validación:** Verificación básica del sistema (ejecución o sanity check)

Si alguna de estas etapas falla, el pipeline debe detenerse.

Merge hacia rama develop:
Una vez validada la funcionalidad, se deberá crear un **merge request** hacia la rama:
- Develop

Durante este proceso:

- Se ejecutan nuevamente las fases de:
  - Build
  - Test

Esto garantiza que el código integrado cumple con los estándares del proyecto.

Despliegue en entorno de desarrollo:
Posterior al merge en develop, se deberá ejecutar automáticamente:

- Construcción de imágenes Docker
- Etiquetado de imágenes
- Despliegue en Kubernetes en el namespace de desarrollo (dev)

En este punto:

- Los cambios deben ser visibles en el entorno de desarrollo
- No deben afectar el entorno productivo

Fase de entrega (Delivery / Release):
La fase de entrega se ejecuta en ramas de tipo:

- release/<version>

o mediante la creación de un **tag**:

- vX.X.X

Jobs obligatorios

1. **Generación de versión**
   - Registro formal de la versión del sistema
2. **Build y publicación de imágenes**
   - Construcción de imágenes Docker finales
   - Publicación en un registry de contenedores

Estas imágenes serán las utilizadas en producción.

Despliegue Continuo (CD) a producción
El despliegue a producción se realizará únicamente al hacer merge hacia:
main / master
En esta etapa:

- El pipeline utilizará las imágenes previamente generadas
- Se actualizarán los deployments en Kubernetes
- Se ejecutará un **rolling update**

Objetivo

- Garantizar alta disponibilidad
- Minimizar interrupciones del servicio

Autoescalado (obligatorio)
Cada microservicio deberá implementar:

- **Horizontal Pod Autoscaler (HPA)**

Condición de escalado

- Cuando el uso de CPU alcance el **80%**, se deberá crear un nuevo pod

Consideraciones de Kubernetes
El sistema deberá cumplir con:

- Arquitectura de microservicios (un contenedor por servicio)
- Configuración de recursos:
  - CPU
  - Memoria

Buenas prácticas obligatorias

- Uso de imágenes versionadas (no usar latest)
- Automatización completa del pipeline
- No realizar despliegues manuales
- Validar el sistema antes de promover a producción
- Mantener separación entre entornos (dev / prod)

Restricciones importantes

- No se permite omitir etapas del pipeline
- No se permite despliegue manual en producción
- No se permite usar imágenes no versionadas
- El pipeline debe ser completamente funcional

4.2 Alcance del proyecto

Alcance obligatorio:

- Extensión funcional de los 5 microservicios existentes
- Implementación de pipeline **CI/CD completo**
- Despliegue del sistema en la nube
- Uso de **Terraform** para provisionamiento
- Bases de datos en **máquinas virtuales (VMs)**
- Despliegue en Kubernetes
- Implementación de **autoescalado (HPA)**
- Actualización completa de documentación

4.3 Recursos y herramientas a utilizar

Tipo	Categoría	Descripción
Obligatorio	Cloud	AWS, GCP o Azure
Obligatorio	IaC	Terraform
Obligatorio	CI/CD	GitHub Actions / GitLab CI / similar
Obligatorio	Contenedores	Docker

Obligatorio	Orquestación	Kubernetes
Obligatorio	Mensajería	Kafka / RabbitMQ / NATS
Obligatorio	Base de datos	Ejecutadas en VM

4.4 Entregables

A continuación se detalla cada uno de los elementos que deberá presentar el estudiante

Tipo	Descripción
Código	Microservicios actualizados
Infraestructura	Scripts Terraform
Pipeline	CI/CD funcional
Despliegue	Sistema en nube
Documento	Arquitectura actualizada
Frontend	Se deberá desplegar el frontend en una instancia virtual o un servicio despliegue rápido en la nube, como Google cloud run. No se aceptara despliegues en S3.

5. Material de apoyo

Categoría	Link	Descripción

Documentación oficial	https://developer.hashicorp.com/terraform/docs	Terraform para infraestructura como código
Documentación oficial	https://kubernetes.io/docs/home/	Kubernetes en entornos productivos
Documentación oficial	https://docs.github.com/en/actions	CI/CD con GitHub Actions
Manual	https://12factor.net/	Buenas prácticas en aplicaciones modernas
Manual	https://sre.google/workbook/table-of-contents/	Prácticas SRE aplicadas

6. Metodología

Pasos a seguir:

1. Paso 1: Análisis del sistema actual
   Evaluar la arquitectura de Fase 1 e identificar puntos de mejora.

2. Paso 2: Extensión de funcionalidades
   Agregar nuevas capacidades a los microservicios existentes.

3. Paso 3: Diseño del pipeline CI/CD
   Definir etapas:
   - Build
   - Test
   - Deploy

4. Paso 4: Implementación del pipeline
   Configurar CI/CD en repositorio.

5. Paso 5: Contenerización
   Optimizar Dockerfiles.

6. Paso 6: Infraestructura con Terraform
   Provisionar:
   - Red
   - VMs
   - Cluster

7. Paso 7: Configuración de bases de datos
   Desplegar DB en máquinas virtuales.

8. Paso 8: Despliegue en Kubernetes
   Configurar deployments, services y autoscaling

9. Paso 9: Pruebas
   Validar sistema completo.

10. Paso : Documentación
    Actualizar todos los artefactos.

Recomendaciones

- Automatizar todo (evitar pasos manuales)
- Usar versionamiento de imágenes
- Validar despliegues progresivos
- Diseñar antes de implementar

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de proyecto	3 de septiembre	3 de septiembre
Elaboración	3 de septiembre	24 de septiembre
Calificación	26 de septiembre	26 de septiembre

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Tema	**Descripción**	**Cumple (Sí/No)**
CI/CD funcional	Pipeline completo implementado	
Terraform funcional	Infraestructura reproducible	
Despliegue en nube	Sistema accesible	
DB en VM	Correctamente implementadas	
Funcionalidades	Implementación de las funcionabilidades descritas en la fase dos del proyecto.	
UEDI	Entrega en UEDI en la fecha estipulada	
Repositorio	Commit antes de la fecha de entrega. Agregar al auxiliar como colaborador.	

8.2 Detalle de la Calificación

	Criterio/Nivel			**Nivel de evaluación**		
No.	**Criterio de evaluación**	**Punteo maximo**		**Satisfactorio 100% - 61%**	**Necesita mejorar 60% - 0%**	**Punteo Obtenido**
1	Habilidades	40				
1.1	Trabajo en equipo	10	Descripción	Distribución clara, integración entre servicios	Trabajo aislado o desbalanceado	
			Rango del punteo	10-5	4-0	
1.2	Documentación técnica	10	Descripción	Documentación actualizada y coherente con Fase 2	Documentación incompleta o desactualizada	
			Rango del punteo	10-5	4-0	

1.3	Justificación técnica	10	Descripción	Decisiones bien argumentadas (CI/CD, Terraform, diseño)	Sin justificación o superficial	
			Rango del punteo	10	0	
1.4	Claridad de arquitectura		Descripción	Diagramas claros y consistentes con cambios	Diagramas confusos o inconsistentes	
			Rango del punteo	10-5	4-0	
	Sub-Total de Puntos					
2	Conocimientos	60				
2.1			Descripción			
			Rango del punteo			

2.2	Implementación de nuevas funcionalidades	5	Descripción	Funcionalidades completas y operativas	Funcionalidades incompletas o inexistentes	
			Rango del punteo	5	0	
2.3	Impacto en el dominio	5	Descripción	Cambios afectan correctamente la lógica del sistema	Cambios superficiales sin impacto real	
			Rango del punteo	5-3	3-0	
2.4	Integración con arquitectura	5	Descripción	Funcionalidades integradas sin romper desacoplamiento	Rompe arquitectura o introduce acoplamiento	
			Rango del punteo	5-3	2-0	
2.5	Nuevos eventos definido	5	Descripción	Eventos claros, estructurados y utilizados	Eventos ambiguos o inexistentes	

			Rango del punteo	5	0	
2.6	Pipeline funcional	5	Descripción	Pipeline completo y automatizado	Pipeline incompleto o manual	
			Rango del punteo	5	0	
2.7	Flujo de ramas y despliegue	5	Descripción	Uso correcto de feature, develop, main y release	Flujo incorrecto o inexistente	
			Rango del punteo	5	0	

2.8	Provisionamiento de infraestructura	5	Descripción	Infraestructura reproducible con Terraform	Configuración manual o incompleta	
			Rango del punteo	5-3	2-0	
2.9	Bases de datos en VM	5	Descripción	DB correctamente desplegadas en VM	DB mal ubicadas o dentro de K8s	
			Rango del punteo	5	0	
2.10	Autoescalado (HPA)	Autoescalado (HPA)	Descripción	Escalado configurado al 80% CPU	No implementado o incorrecto	
			Rango del punteo	5-3	2-0	

	Sub-Total de Puntos					
	Total					

8.3 Comentarios Generales

Se reducirá significativamente la nota si:

- Las nuevas funcionalidades no generan eventos
- Se rompe la comunicación asíncrona
- Se introduce acoplamiento entre servicios
- CI/CD no es funcional
- Terraform no es reproducible
- No se integra con la Saga existente